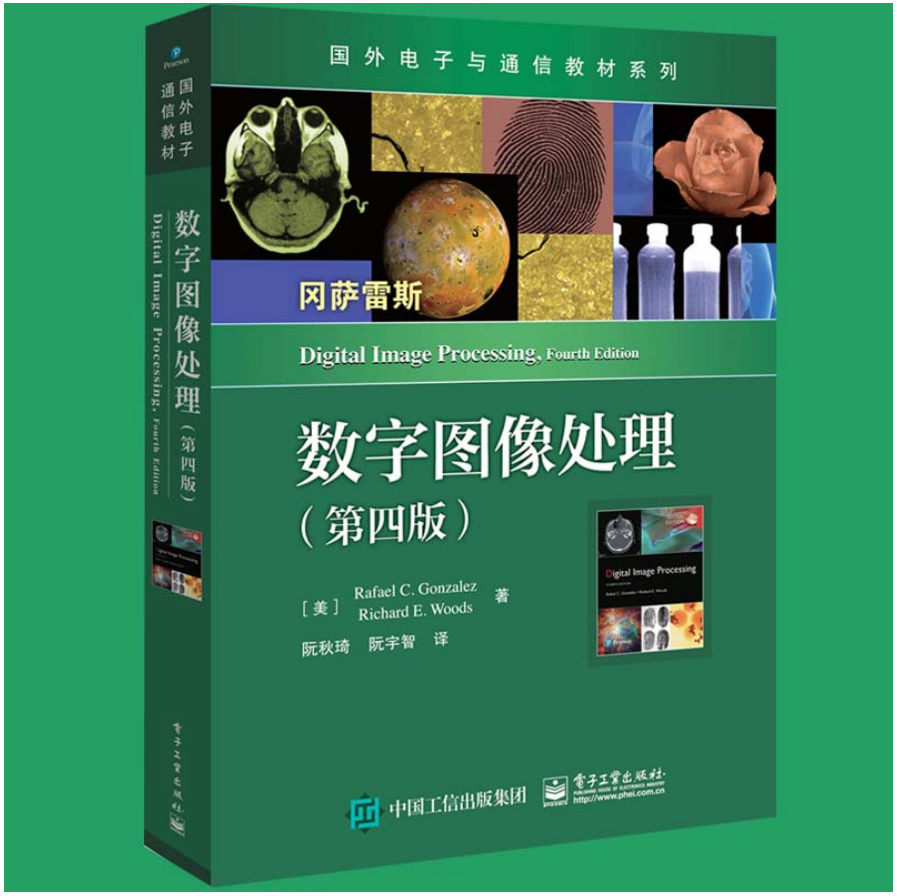
多旋翼飞行器设计与控制实践

视觉基础

全权　杨坤
自动化科学与电气工程学院
北京航空航天大学

2.1 数字图像处理

- 第二部分：视觉基础
  - 数字图像处理
    - OpenCV安装与基本操作
    - 灰度变化与滤波
    - 图像分割
    - 形态学处理
  - 目标检测与跟踪
  - 定位基础总结
  - 总结



2.1 数字图像处理——OpenCV安装与基本操作

1. 安装

   pip install opencv-python

2. imread函数读取数字图像

   cv2.imread(path_of_image, intflag)

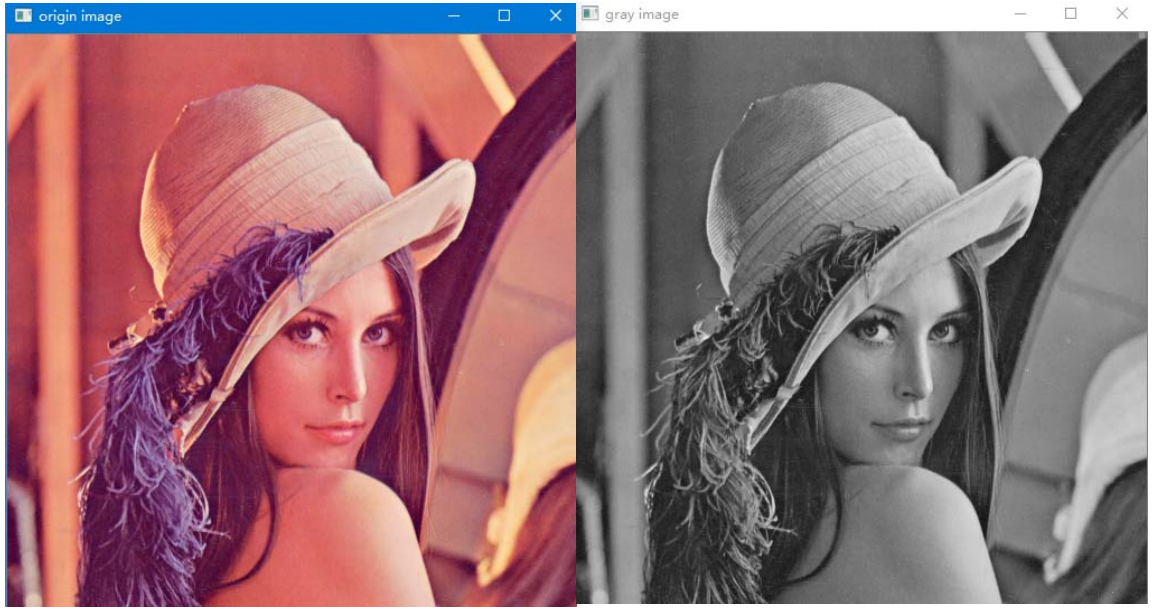
3. imshow函数显示图像

   cv2.imshow(windows_name, image)

   cv2.waitKey(0)

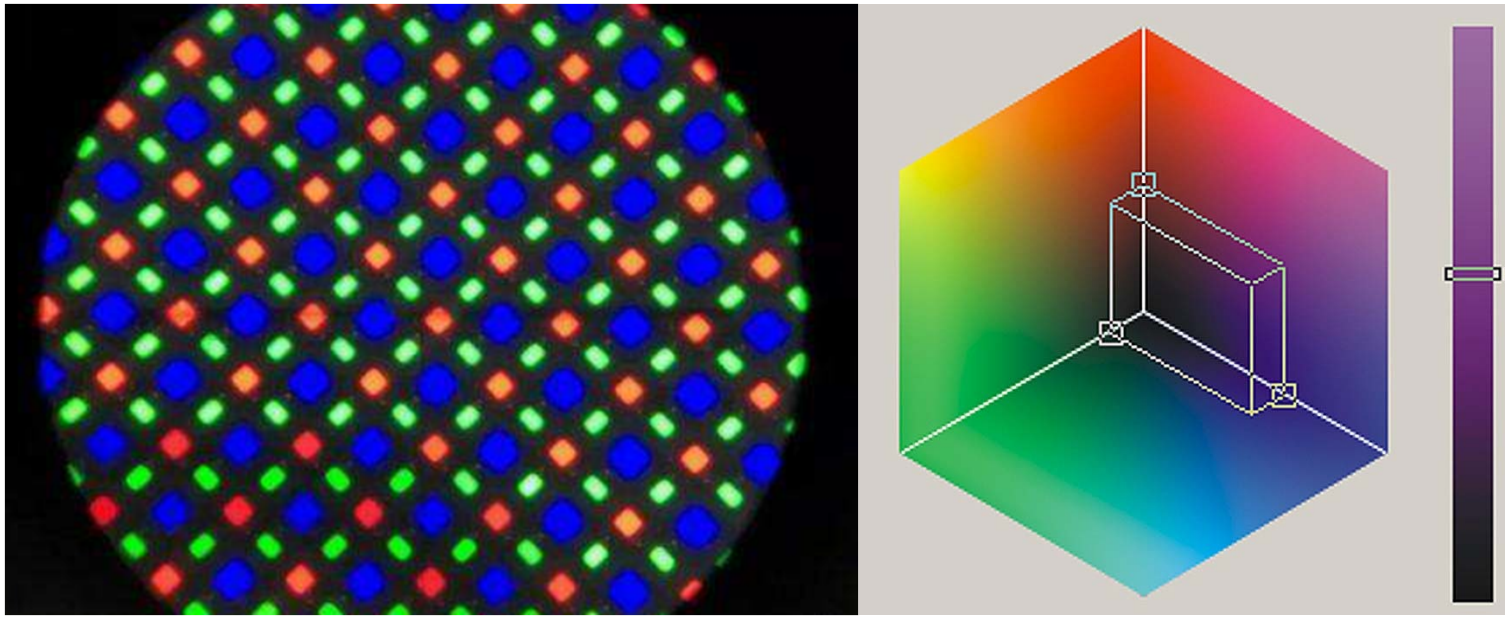
4. imwrite函数写入图像

   cv2.imwrite(image_filename, image)

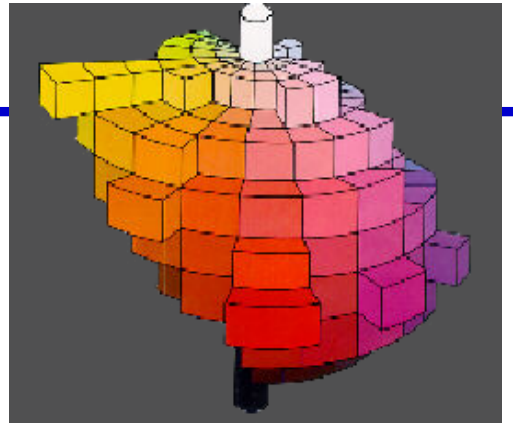


2.1 数字图像处理—— RGB色彩空间

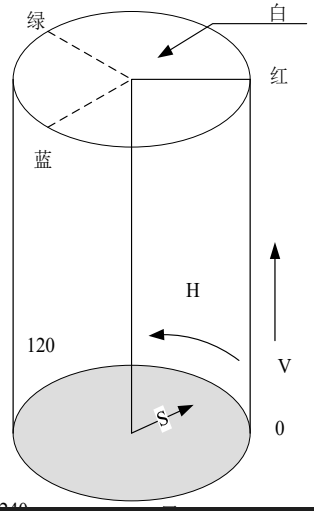
- 计算机显示器用红R、绿G、蓝B光的组合产生颜色，(R,G,B)的值唯一地确定在显示器上的显示颜色， 如（180，70，250）产生一个偏绿的蓝色
- 三种颜色构成了三维空间坐标-- RGB颜色空间



2.1 数字图像处理—— HSV颜色空间



- HSV能减少彩色图像处理的复杂性。
- 合乎人对彩色的认识：
  - 色彩：色调、饱和度
  - 亮度：非彩色属性，对应黑白图像的灰度。
  - 色调和饱和度通称为色度，表示颜色类别与深浅程度。



- RGB到HSV色彩空间转换公式：

$$V=\sqrt{3}(R+G+B)$$

$$S=\frac{\rho}{\rho_{\max}\frac{3\min(R,G,B)}{R+G+B}\frac{\sqrt{3}}{V}\min(}$$

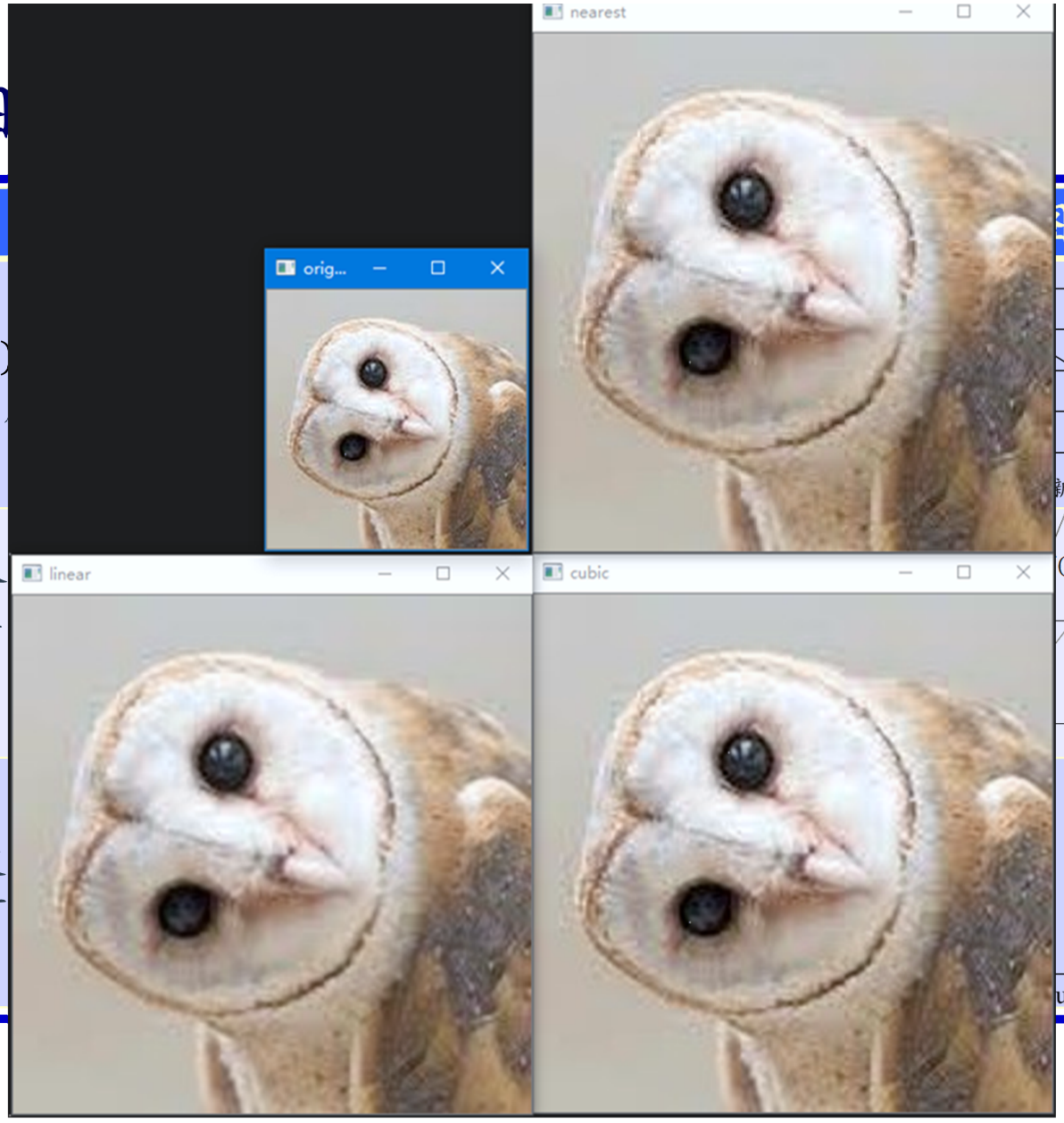
$$\theta=\cos^{-1}\left[\frac{\frac{1}{2}[R-G]+[R-B]}{\sqrt{(R-G)^2+(R-B)(G-B)}}\right]$$

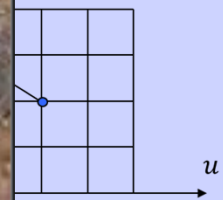
$$H=\begin{cases}\theta & G\geq B\\ 2\pi-\theta & G\leq B\end{cases}$$

```
img_hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
```

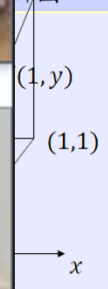
2.1 数字图

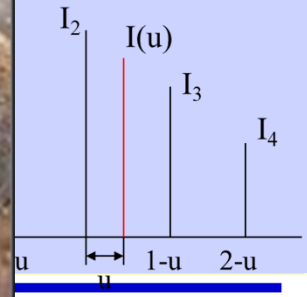
方法	
近邻法	(u,v) 校正
双线性插值	两个 性质 糊
立方卷积插值	三次 列方



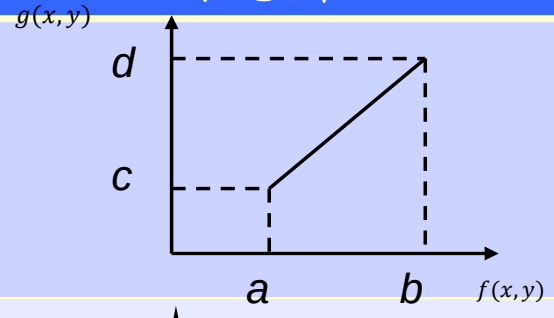
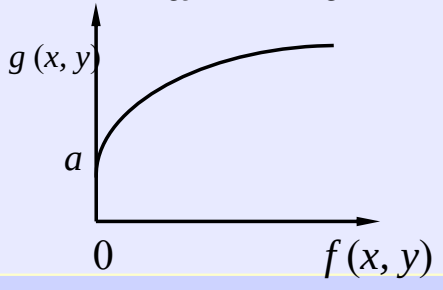
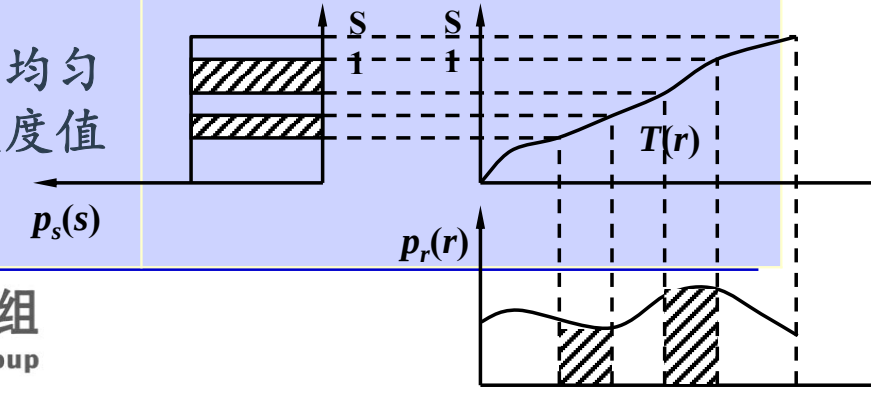


新图





2.1 数字图像处理——灰度变换

方法	原理	示意图
线性变换	$f(x,y)$ 的动态范围 $[a,b]$ → $g(x,y)$ 的动态范围 $[c,d]$	
非线性变换	突出感兴趣的区间，相对抑制不感兴趣的灰度区域	
直方图均衡	把原始图的直方图变换为均匀分布的形式，增加像素灰度值的动态范围	

2.1 数字图像处理——灰度变换

```python
#coding=utf-8
import cv2
import numpy as np

image = cv2.imread("D:/test/unequ.jpg", 0)

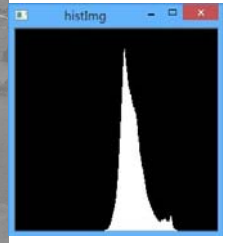
lut = np.zeros(256, dtype = image.dtype )#创建空的查找表

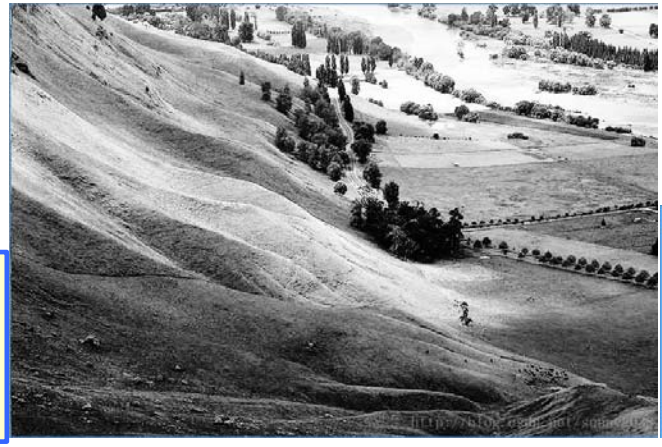
hist,bins = np.histogram(image.flatten(),256,[0,256])
cdf = hist.cumsum() #计算累积直方图
cdf_m = np.ma.masked_equal(cdf,0) #除去直方图中的0值
cdf_m = (cdf_m - cdf_m.min())*255/(cdf_m.max()-cdf_m.min())#等同于前面介
绍的lut[i] = int(255.0 *p[i])公式
cdf = np.ma.filled(cdf_m,0).astype('uint8') #将掩模处理掉的元素补为0

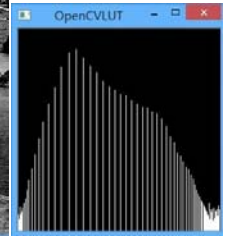
#计算
result2 = cdf[image]
result = cv2.LUT(image, cdf)

cv2.imshow("OpenCVLUT", result)
cv2.imshow("NumPyLUT", result2)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



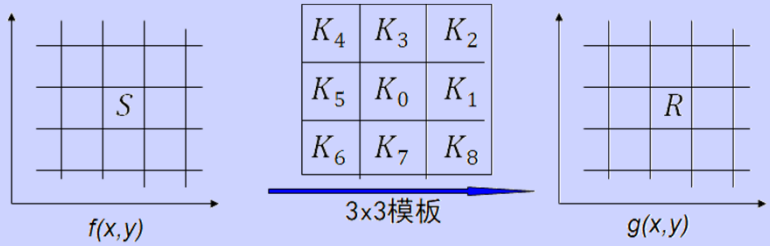
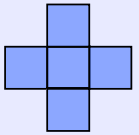
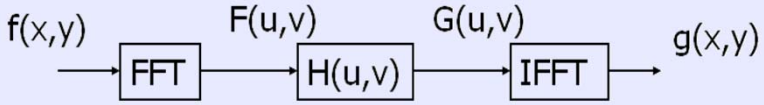
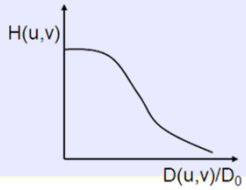






结论：直方图均衡是一种非线性变换，以牺牲图像等级为代价，增加像素灰度值的动态范围，提高图像对比度。

2.1 数字图像处理——图像平滑

方法	原理	示意图
邻域平均法	$g(x,y)=\sum_{i=-M}^{M}\sum_{j=-M}^{M}h(i,j)f(x+i,y+j)=h*f$	
中值滤波	取某一窗口中各点值排序，并取位置居中的值为该点的像素灰度值	
多图像平均法	把原始图的直方图变换为均匀分布的形式，增加像素灰度值的动态范围	
频域平滑		

2.1 数字图像处理——同态增晰

分析：关心反射信
均，能否消除照度

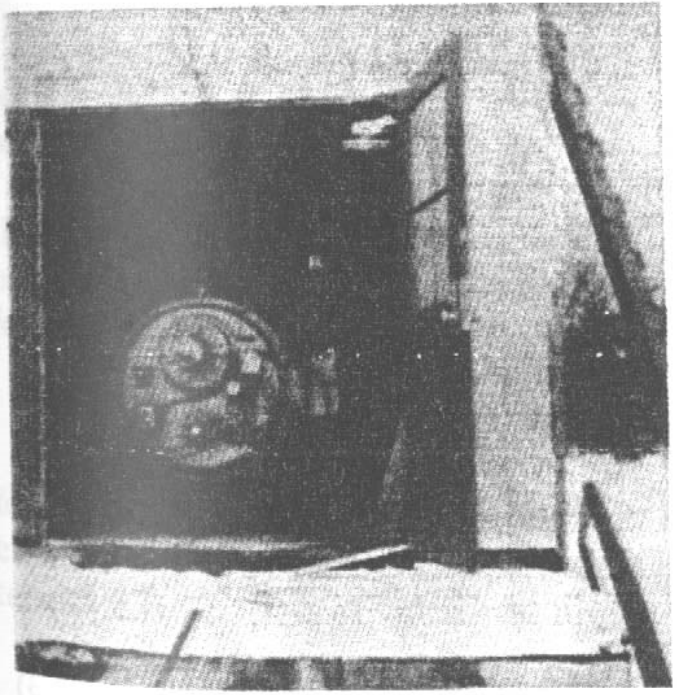
过程：

$f(x,y)$ → ln

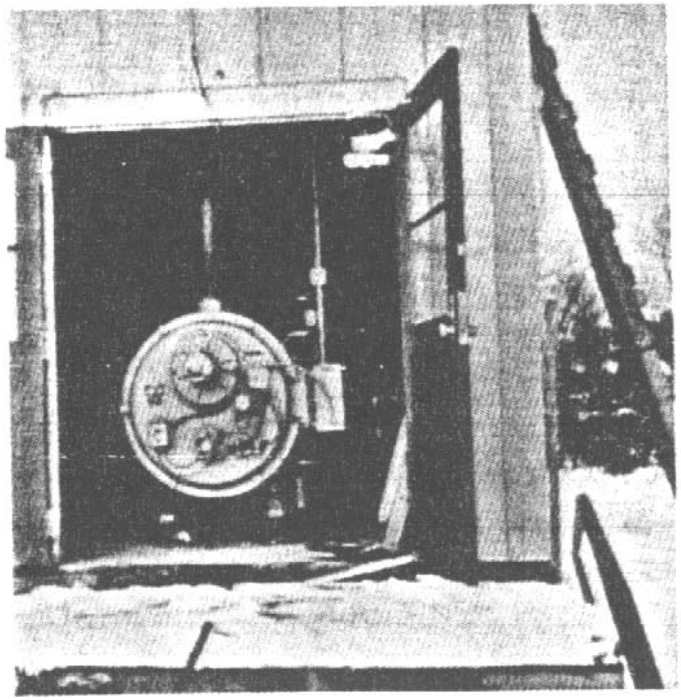
$f(x,y) = i(x,y)r(x,y$

$i(x,y)$—照射分

$r(x,y)$—反射分



(a)

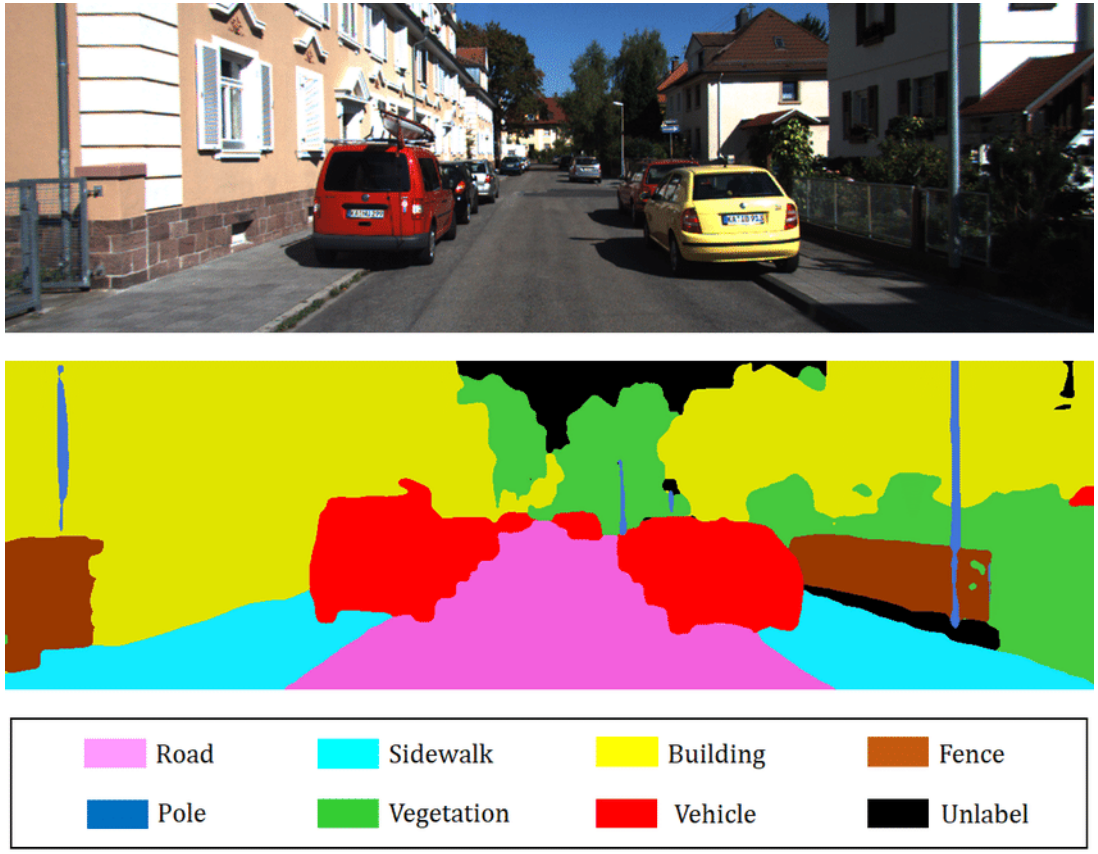


(b)

2.1 数字图像处理——图像分割

图像分割：将图像中有意义的特征或需要应用的特征提取出来。

- 阈值分割
- 边缘检测
- 区域分割
- Hough变换
- 近邻法分割



2.1 数字图像处理——最大方差阈值选取法

现在用阈值T分开：当z>T 时为背景，反之则是对象物。使得误差最小

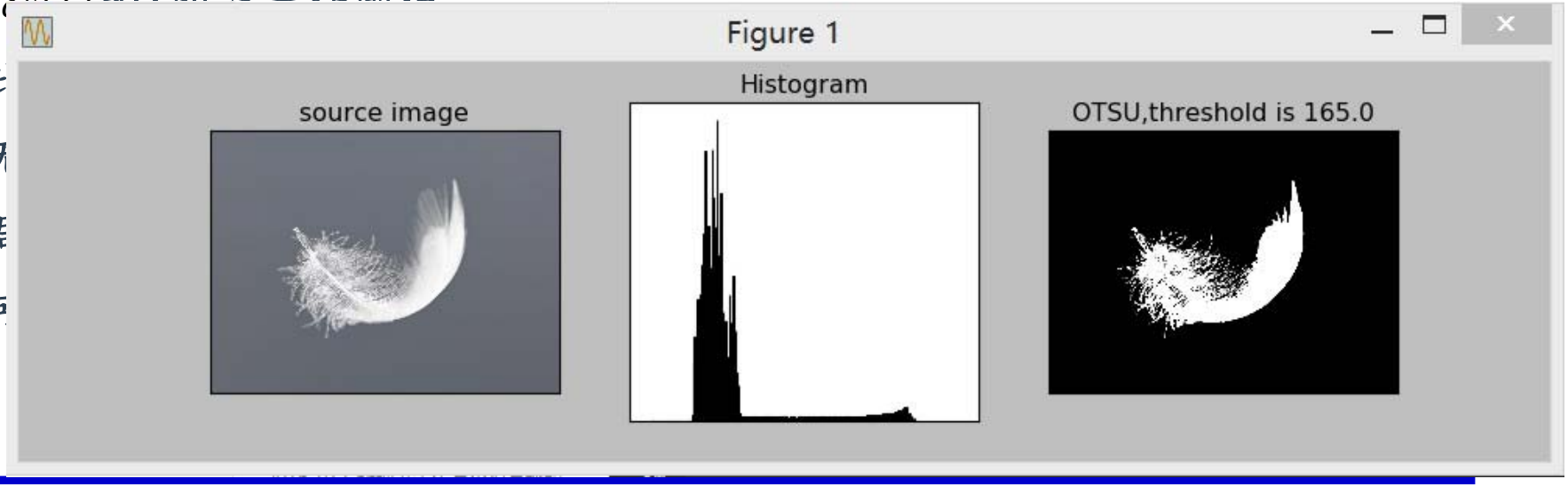
$$tp(z) + (1-t)q(z)$$

实现过程：从T从1~m逐一改变，每变一

大

* 此
* 开
* 后
* 可

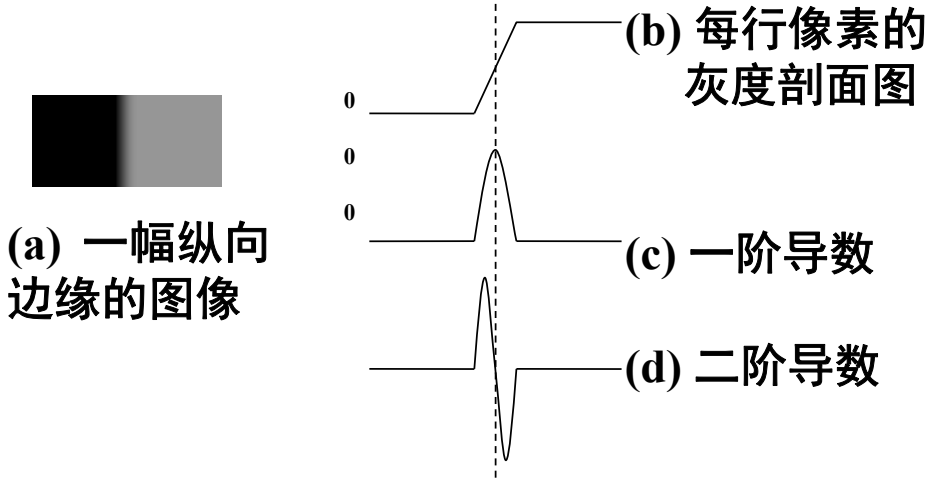
```
ret1, th1 = cv2.threshold(gray, 0, 255, cv2.THRESH_OTSU)
```



2.1 数字图像处理——边缘检测

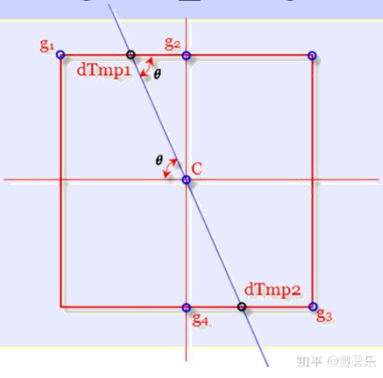
◘ 边缘检测原理

其导数在边缘方向取得极值



从数学上看，图像的模糊相当于图像被平均或积分，为实现图象的锐化，必需用它的反运算“微分”——加强高频分量作用，使轮廓清晰。

2.1 数字图像处理——边缘检测

方法	原理	示意图
Sobel算子	典型的基于一阶导数的边缘检测算子	$G_x=\begin{bmatrix}-1 & 0 & 1\\ -2 & 0 & 2\\ -1 & 0 & 1\end{bmatrix}, G_x=\begin{bmatrix}1 & 2 & 1\\ 0 & 0 & 0\\ -1 & -2 & -1\end{bmatrix}$
Roberts算子	根据任意一对互相垂直方向上的差分来计算梯度。在水平和垂直方向上效果较好，但是对噪声敏感。	$G_x=\begin{bmatrix}1 & 0\\ 0 & -1\end{bmatrix}, G_y=\begin{bmatrix}0 & 1\\ -1 & 0\end{bmatrix}$
Laplace算子	各向同性算子，二阶微分算子，在只关心边缘的位置而不考虑其周围的象素灰度差值时比较合适。	$\begin{bmatrix}0 & 1 & 0\\ 1 & -4 & 1\\ 0 & 1 & 0\end{bmatrix}$
Canny算子	**step1:** 用高斯滤波器平滑图象； **step2:** 用一阶偏导的有限差分来计算梯度的幅值和方向； **step3:** 对梯度幅值进行非极大值抑制 **step4:** 用双阈值算法检测和连接边缘	

区域分割：利用的是图像的空间性质，认为分割出来的属于同一区域的像素应具有相似的性质.

1. 区域生长法：

1	0	4	7	7
1	0	4	7	7
0	1	5	5	5
2	0	5	6	5
2	2	5	6	4

1	1	5	5	5
1	1	5	5	5
1	1	5	5	5
1	1	5	5	5
1	1	5	5	5

T=3时的生长结果

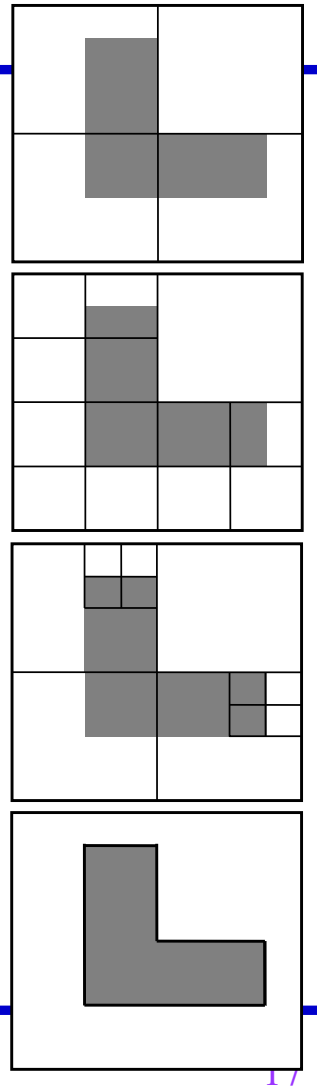
2.1 数字图像处理——区域分割

1. 区域生长法：

- Step1：对像素进行扫描，找出尚没有归属的像素；
- Step2：以该像素为中心检查它的邻域像素，即将邻域中的像素逐个与它比较，如果灰度差小于预先确定的阈值T，将它们合并；
- Step3：以新合并的像素为中心，返回到步骤2，检查新像素的邻域，直到区域不能进一步扩张；
- Step4：返回到步骤1，继续扫描直到所有像素都有归属，则结束整个生长过程。

1	0	4	7	7
1	0	4	7	7
0	1	5	5	5
2	0	5	6	5
2	2	5	6	4

1	1	5	5	5
1	1	5	5	5
1	1	5	5	5
1	1	5	5	5
1	1	5	5	5



2 分裂合并法

实际中常先把图像分成任意大小且不重叠的区域，然后再合并或分裂这些区域以满足分割的要求,即分裂合并法.一致性测度可以选择基于灰度统计特征(如同质区域中的方差),假设阈值为T ,则算法步骤为:

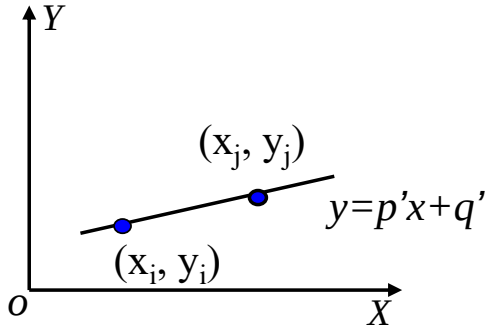
① 对于任一R_i，如果 $V(R_i) > T$ ，则将其分裂成互不重叠的四等分;

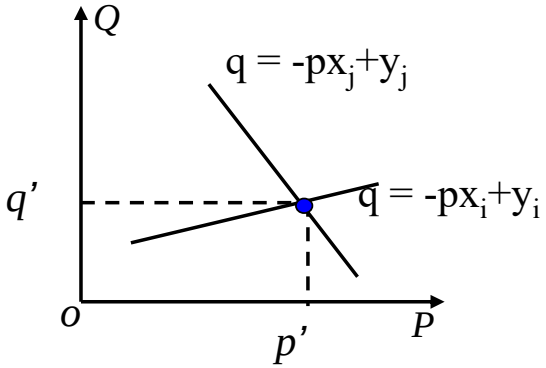
② 对相邻区域R_i和R_i，如果 $V(R_i \cup R_j) \le T$ ，则将二者合并;

③ 如果进一步的分裂或合并都不可能了，则终止算法。

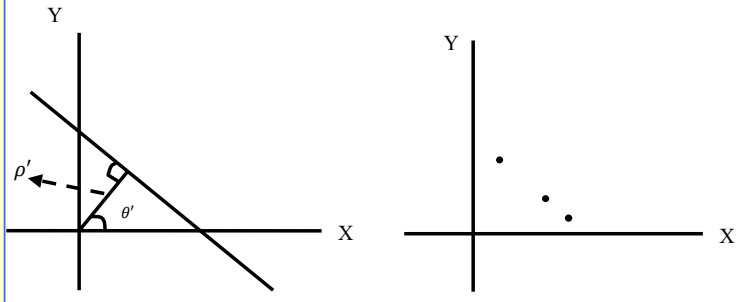
2.1 数字图像处理——霍夫变换

Hough变换是一种检测、定位直线和解析曲线的有效方法。它是把二值图变换到Hough参数空间，在参数空间用极值点的检测来完成目标的检测。下面以直线检测为例，说明Hough变换的原理。





图像空间和参数空间中点和线的对偶性



$$\rho = x_0 \cos\theta + y_0 \sin\theta \quad = A\sin(\alpha + \theta)$$

2.1 数字图像处理——霍夫变换

Hough变换检测二次曲线---以圆为例

圆的方程： $(x-a)^2+(y-b)^2=r^2$

参数坐标： $(a-x)^2+(b-y)^2=r^2$

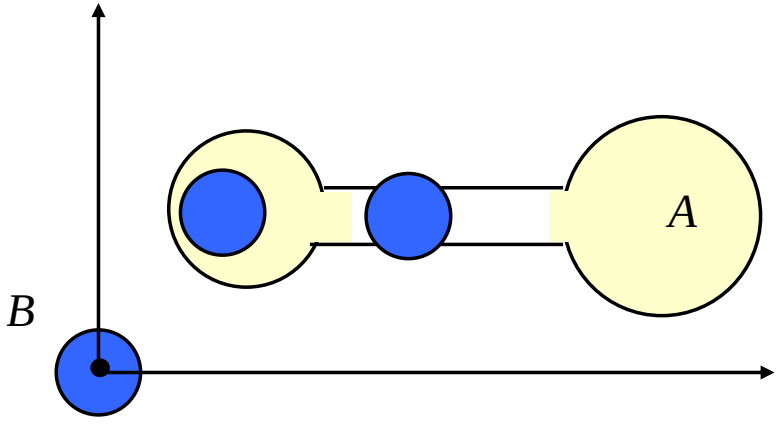
Hough变换的优点是抗噪声能力强，能够在信噪比较低的条件下，检测出直线或解析曲线。缺点是需要首先做二值化以及边缘检测等图像预处理工作，损失掉原始图像中的许多信息。



形态学研究几何结构的基本思想

利用一个结构元素（相当于模板）去探测一个图像，看是否能将这个结构元素很好地填放在图像的内部，同时验证填放结构元素的方法是否有效。

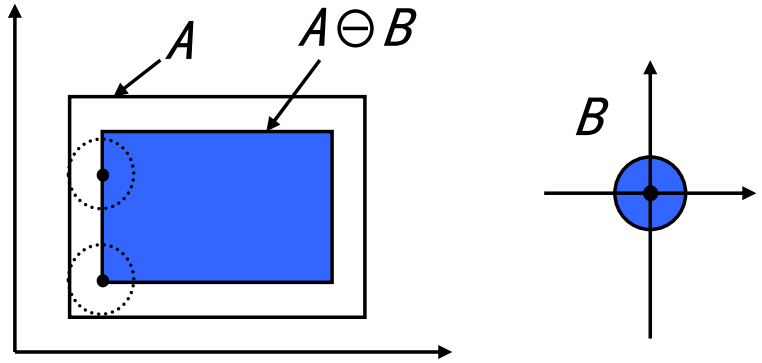
对图像内适合放入结构元素的位置做标记，得到关于图像结构的信息。这些信息与结构元素的尺寸和形状都有关。构造不同的结构元素，便可完成不同的图像分析，得到不同的分析结果。



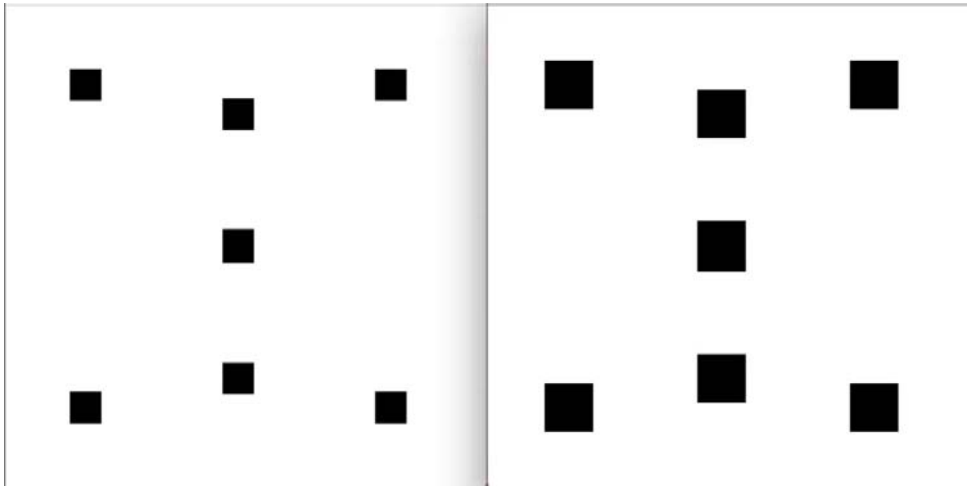
形态学基本运算

腐蚀

$A \ominus B = \{x : B + x \subset A\}$ 表示将 B 平移 x 但仍包含在A 内的所有点 x 组成. 若把 A 看作输入图像, B 看作模板, 则$A \ominus B$ 由在平移模板的过程中, 所有可以添入 A 内部的模板的原点组成.

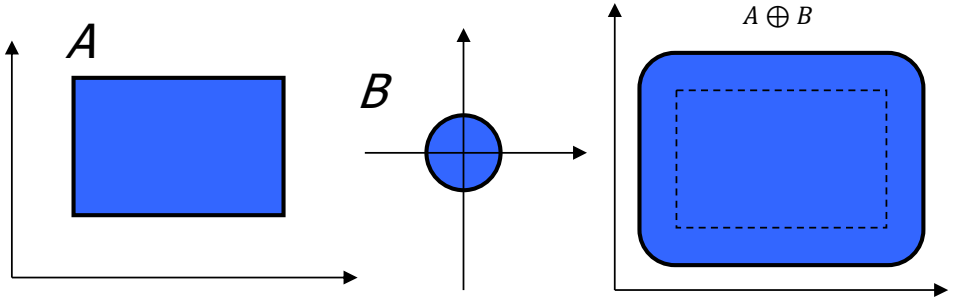


腐蚀类似于收缩

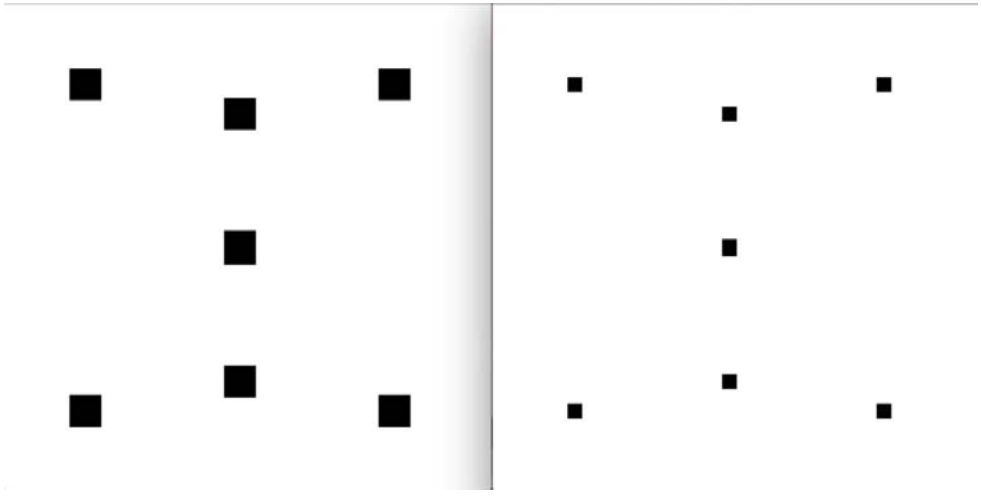


膨胀

$A \oplus B = [A^C \ominus (-B)]^C$ 对于圆盘状结构元素，膨胀可以填充图象内部的小孔，及在图象边缘处的小凹陷部分，并能够磨平图象向外的尖角。



利用圆盘膨胀

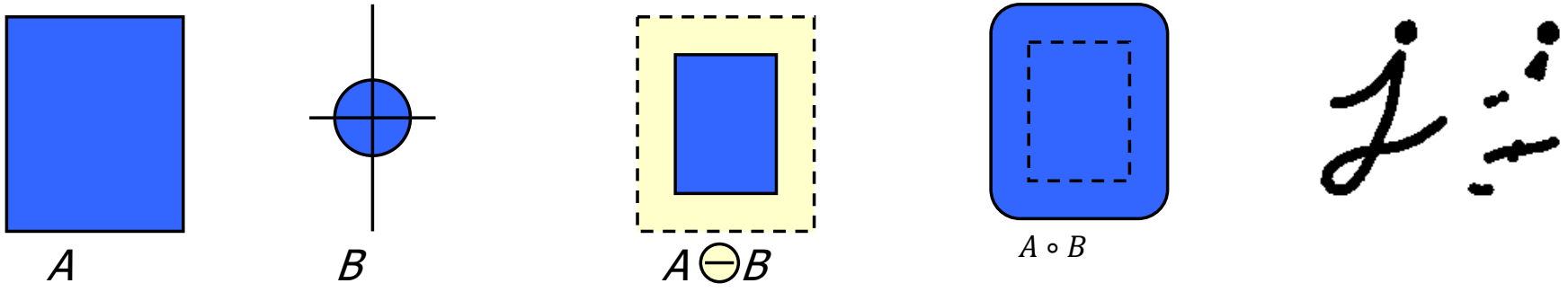


开运算

利用图像 B 对图像 A 做开运算，用符号 $A \circ B$ 表示，其定义为：

$$A \circ B = (A \ominus B) \oplus B$$

开运算的结果删除了不能包含结构元素的对象区域，平滑了对象的轮廓，断开了狭窄的连接，去掉了细小的突出部分

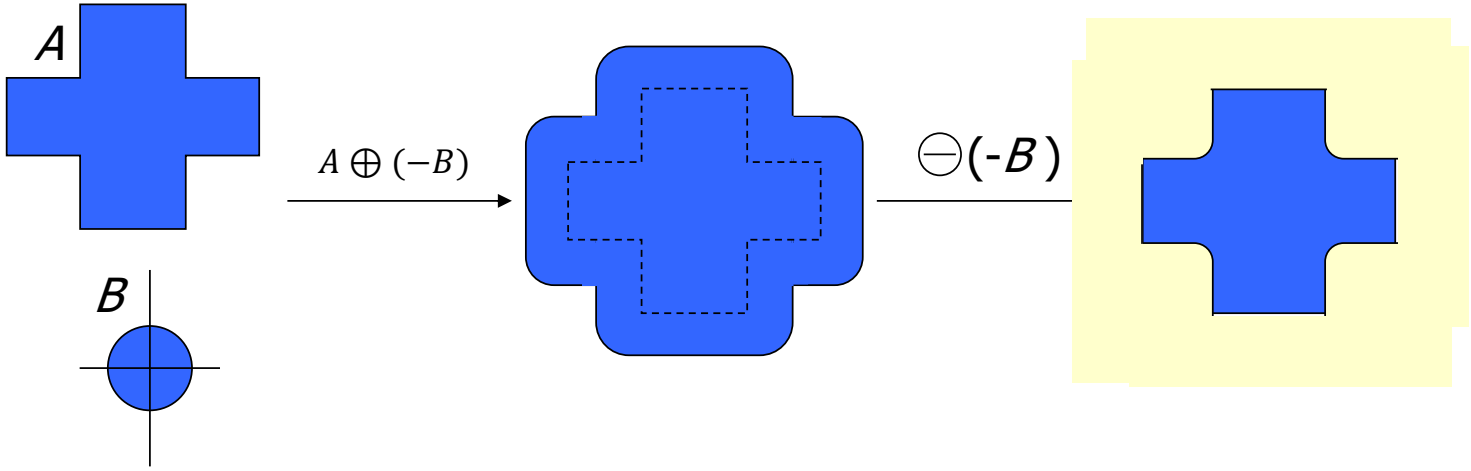


闭运算

是开运算的对偶运算，定义为先作膨胀然后再作腐蚀. 利用 B 对 A 作闭运算表示为 $A \bullet B$，其定义为：

$$A \bullet B = [A \oplus (-B)] \ominus (-B)$$

闭运算一般会将狭窄的缺口连接起来形成细长的弯口，并填充比结构元素小的洞。



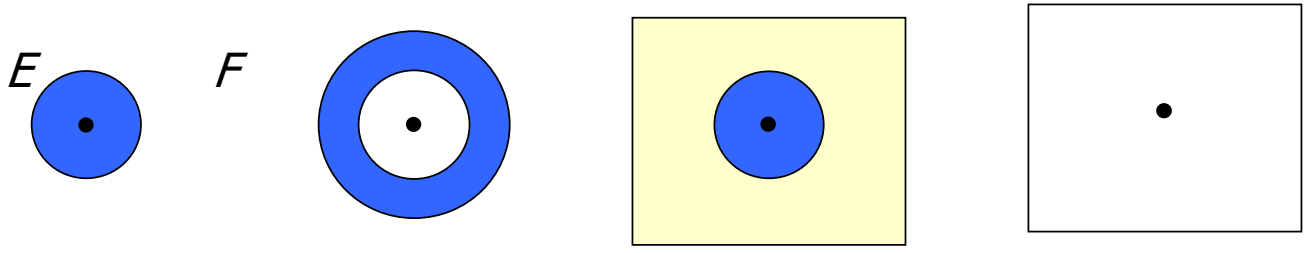


2.1 数字图像处理——击中击不中

定义：击中击不中变换需要两个结构基元E和F，这两个基元被作为一个结构元素对B=(E，F)，一个探测图像内部，一个探测图像外部，其定义为：

$$A * B = (A \ominus E) \cap (A^C \ominus F)$$

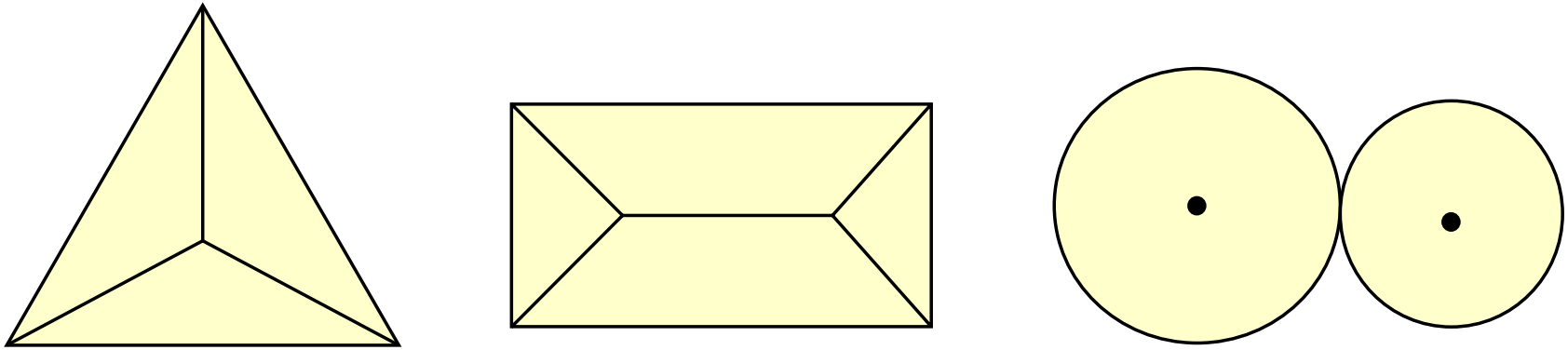
显然 $E \cap F = \varphi$



击中结构元素　　击不中结构元素　　输入图像　　击中击不中输出

2、骨架化

骨架：所有最大圆盘的圆心构成了图像的骨架。



最大圆盘定义的骨架

- 第二部分：视觉基础
  - 数字图像处理
  - 目标检测与跟踪
    - ➢目标检测
    - ➢单目标跟踪
  - 定位基础总结
  - 总结

2.2 目标检测与跟踪——参数指标

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$



如果以 W 代表 A 和 B 的交集 C 的 x 轴方向上的边长，那么有

$$W = max(min(A.x1, B.x1) - max(A.x0, B.x0), 0)$$

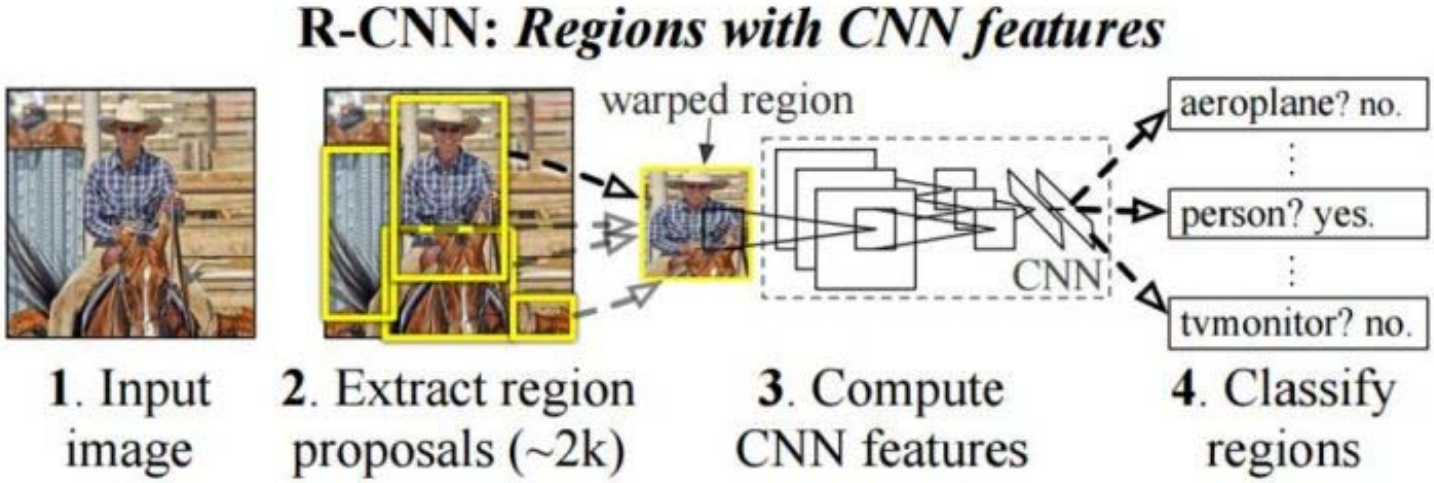
$$H = max(min(A.y1, B.y1) - max(A.y0, B.y0), 0)$$

2.2 目标检测与跟踪—— R-CNN

两阶段模型因其对图片的两阶段处理得名，也称为基于区域（Region-based）的方法，我们选取R-CNN系列工作作为这一类型的代表。

1）CNN可用于基于区域的定位和分割物体；

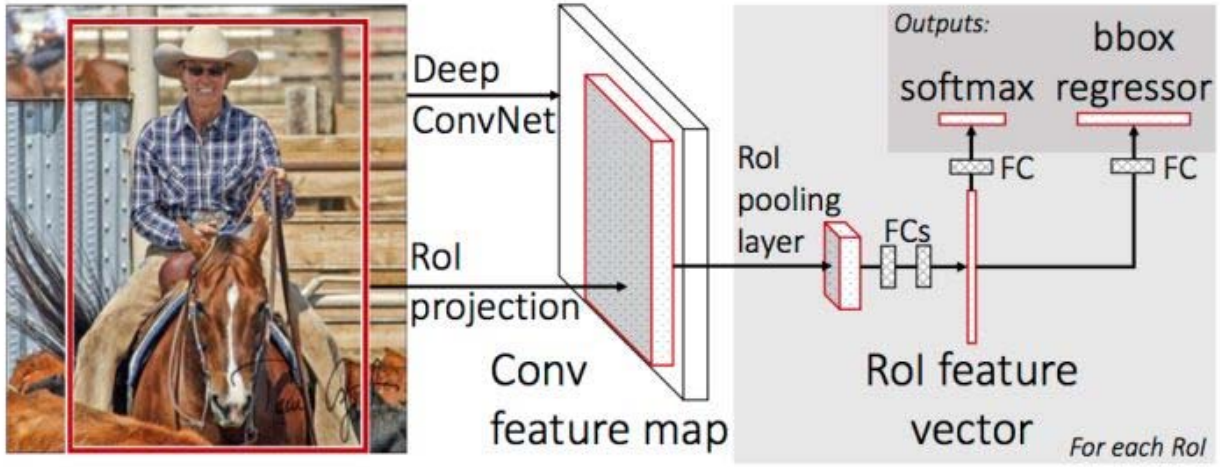
2）监督训练样本数紧缺时，在额外的数据上预训练的模型经过fine-tuning可以取得很好的效果。



2.2 目标检测与跟踪—— Fast R-CNN

□ Fast R-CNN：共享卷积运算

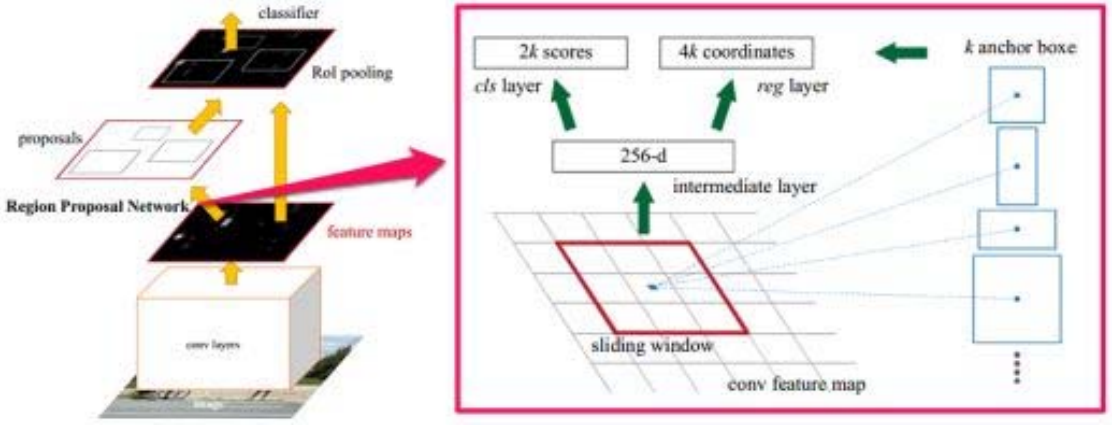
R-CNN耗时的原因是CNN是在每一个Proposal上单独进行的，没有共享计算，便提出将基础网络在图片整体上运行完毕后，再传入R-CNN子网络，共享了大部分计算，故有Fast之名。



2.2 目标检测与跟踪—— Faster R-CNN

Faster R-CNN: 两阶段模型的深度化

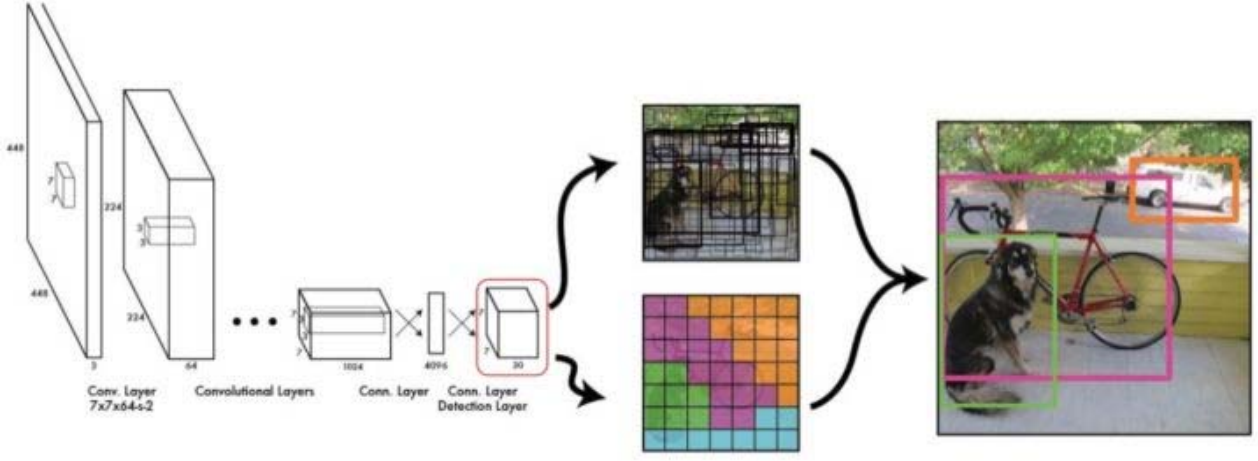
Faster R-CNN是2-stage方法的奠基性工作，提出的RPN网络取代Selective Search算法使得检测任务可以由神经网络端到端地完成。粗略的讲，Faster R-CNN = RPN + Fast R-CNN，跟RCNN共享卷积计算的特性使得RPN引入的计算量很小，使得Faster R-CNN可以在单个GPU上以5fps的速度运行，而在精度方面达到SOTA（State of the Art，当前最佳）。



2.2 目标检测与跟踪—— YOLO

YOLO是单阶段方法的开山之作。它将检测任务表述成一个统一的、端到端的回归问题，并且以只处理一次图片同时得到位置和分类而得名。YOLO的主要优点：

- 快。
- 全局处理使得背景错误相对少，相比基于局部（区域）的方法， 如Fast RCNN。
- 泛化性能好，在艺术作品上做检测时，YOLO表现比Fast R-CNN好。



2.2 目标检测与跟踪—— YOLO

1. 配置环境：安装显卡对应版本的CUDA；下载源码git clone https://github.com/pjreddie/darknet；编译。

编译完成可以用作者的预训练模型测试，正常的话会输出图片的检

```
wget https://pjreddie.com/media/files/yolov3.weights
./darknet detect cfg/yolov3.cfg yolov3.weights data/dog.jp
```

2. 使用yolo_mark标注

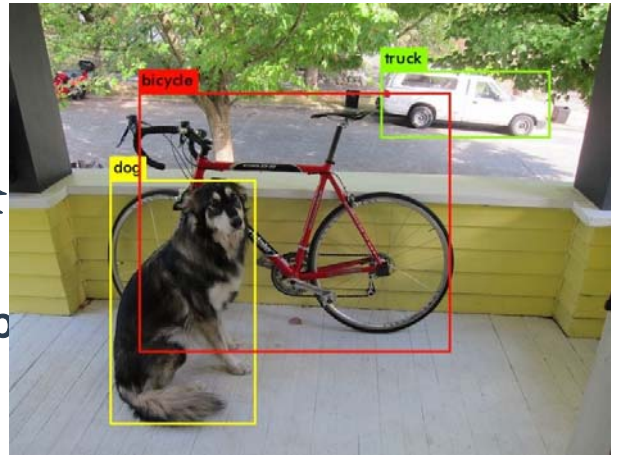
```
https://github.com/AlexeyAB/Yolo_mark.git
```

3. 训练darknet

```
./darknet detector train data/dropnet.data cfg/dropnet.cfg
weights/darknet19_448.conv.23
```

4. 使用

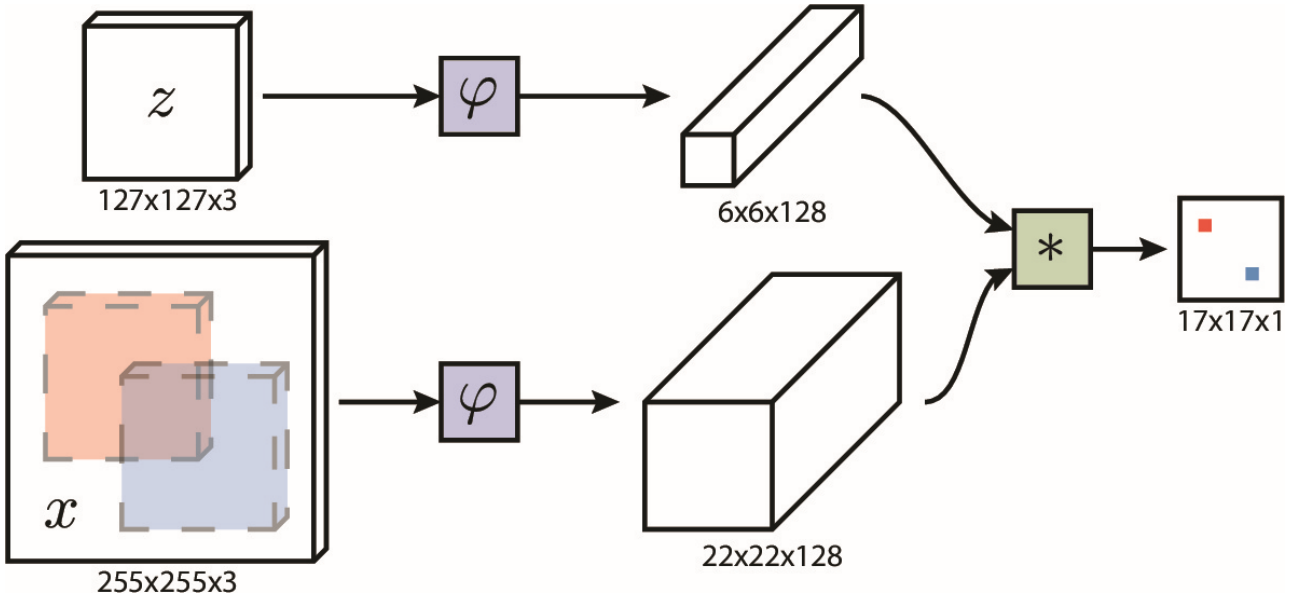
```
./darknet detector test ./data/dropnet.data ./cfg/dropnet.
./weights/dropnet_final.weights -thresh 0.1 -ext_output PA
```





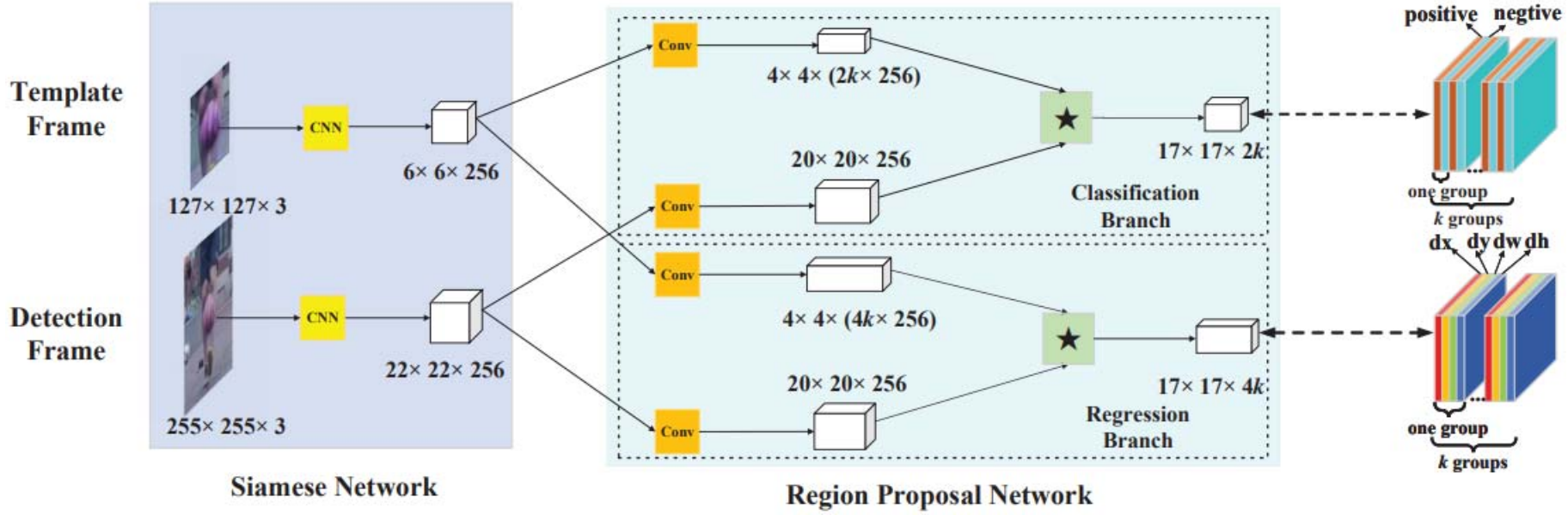
2.2 目标检测与跟踪—— SiamFC

关于SOT(single object track)，我们可以把当做一个配对问题，即把第一帧的目标当做模板，去匹配其他帧。基于这种思路，网络并不需要“理解”目标，只需当新的一帧图像来到时，拿模板找相同即可



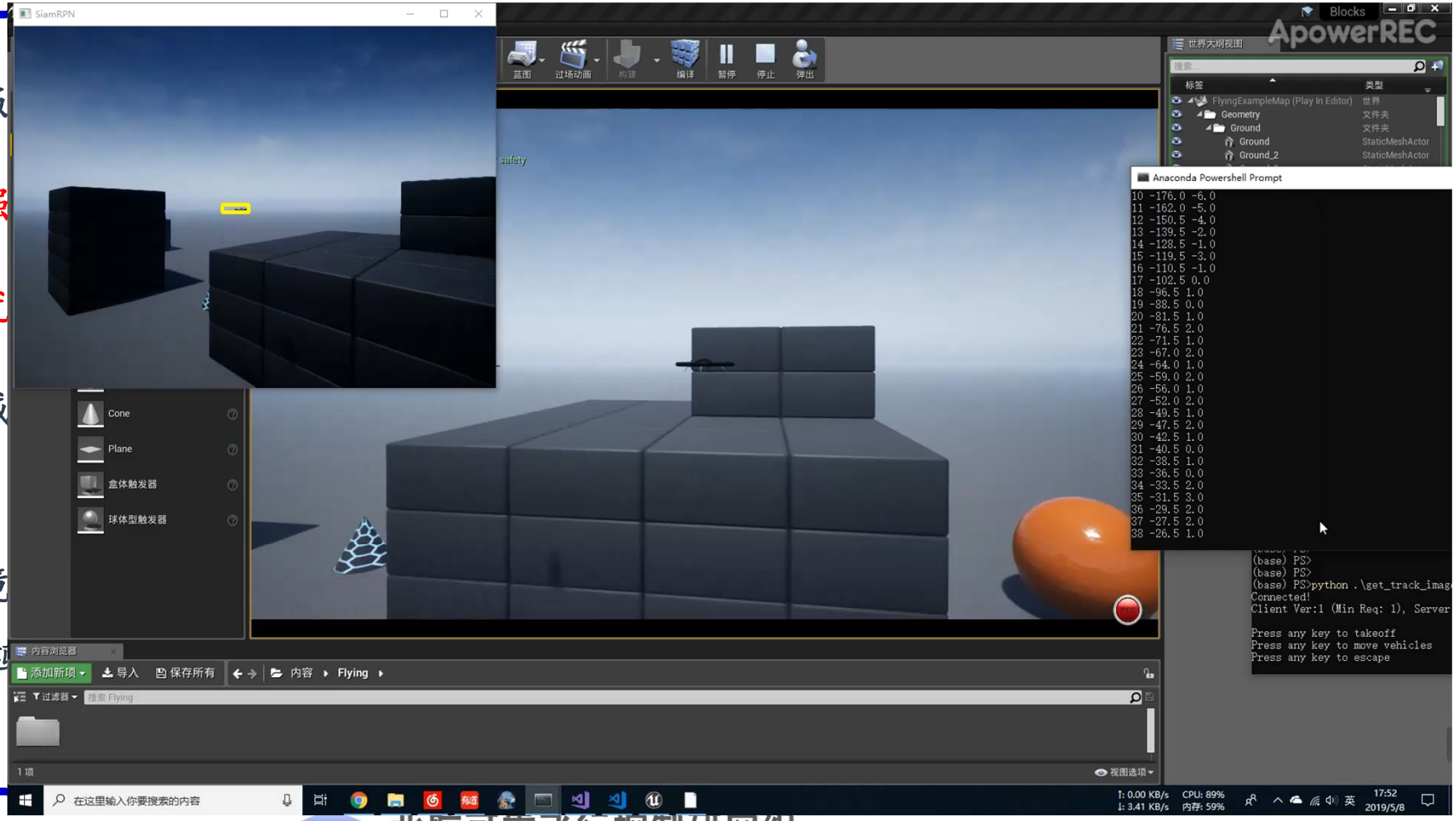
2.2 目标检测与跟踪—— SiamRPN

SiamRPN是CASIA在2018提出来的网络，它将Siam与检测领域的RPN进行结合。



2.2 目标检测与跟踪—— DaSiamRPN

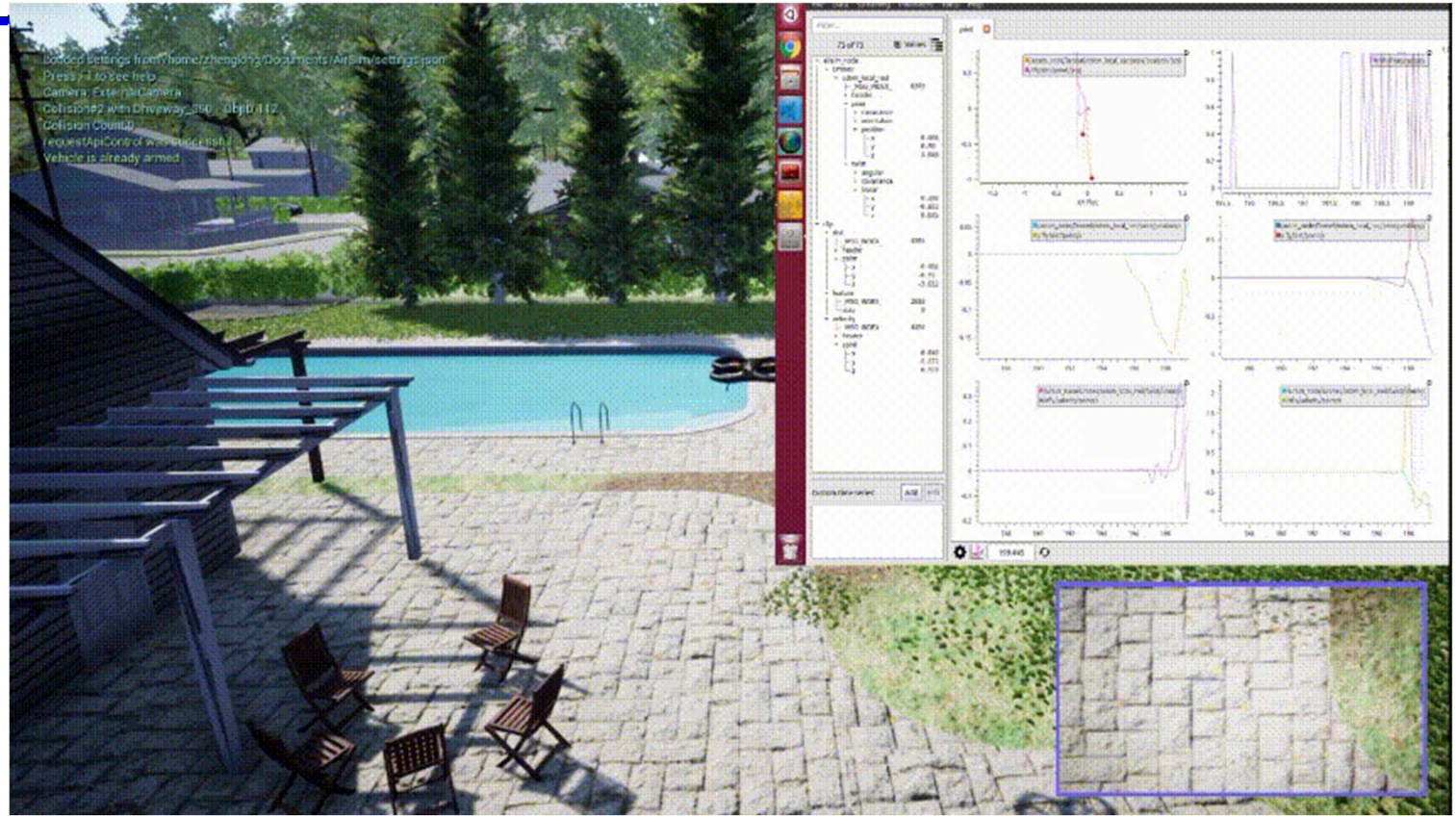
- 较前一版
  1. 数据增强
  2. 学习干扰
  3. 搜索区域
- 使用：
  1. 配置环境
  2. 修改代码



- 第二部分：视觉基础
  - 数字图像处理
  - 目标检测与跟踪
  - 几种定位方法
    - ➢基于全局光流的无人机速度估计
    - ➢**VIO/SLAM**
  - 总结

2.3 几种定位方法:基于全局光流的无人机速度估计

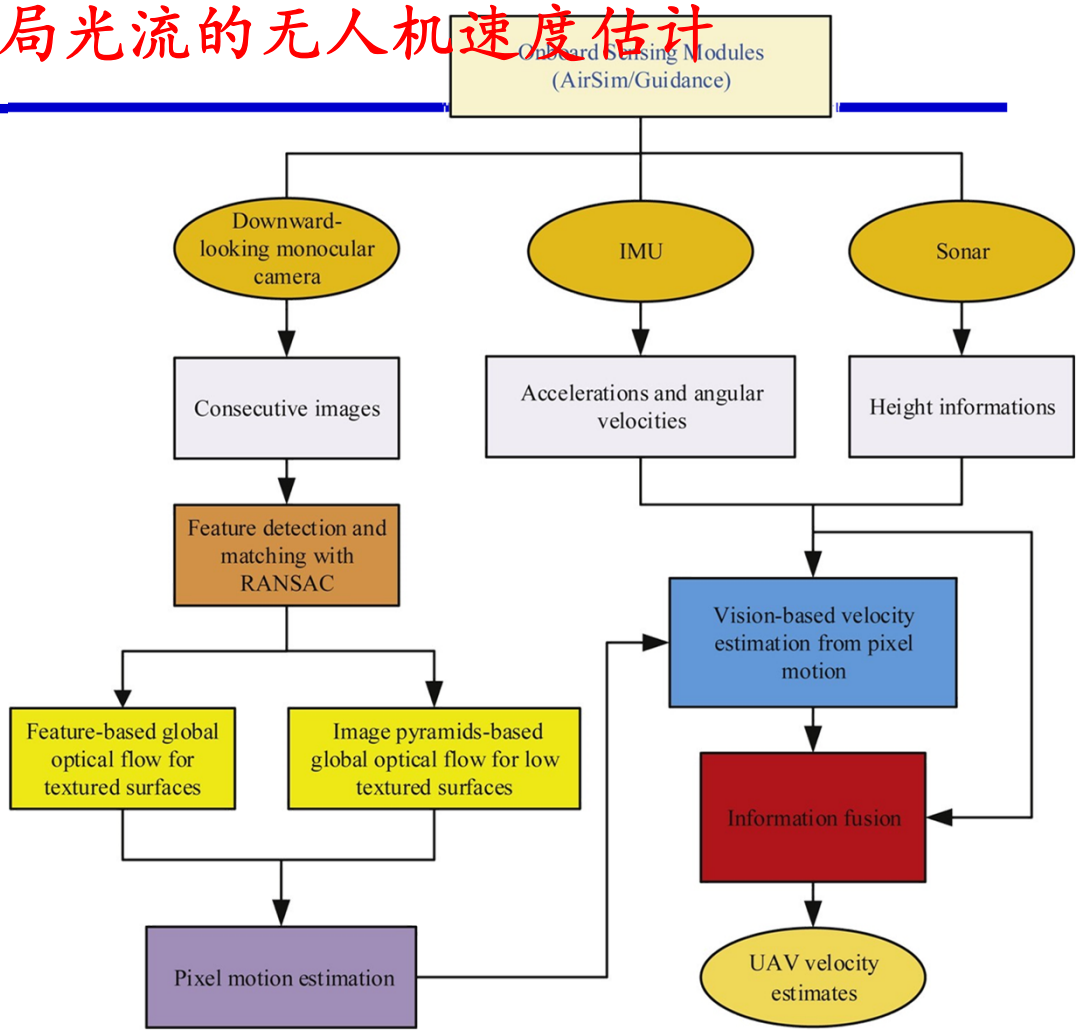
首先，提出了一种基于特征的丰富纹理表面估计方法，并将基于图像金字塔的方法扩展到低纹理背景中以获得像素运动。然后，结合像素运动估计，提出了基于视觉的速度估计方法，给出了视觉观测结果。在此基础上，提出了一种经典的线性卡尔曼滤波算法。



Deng H, Arif U, Yang K, et al. Global optical flow-based estimation of velocity for multicopters using monocular vision in GPS-denied environments[J]. Optik, 2020: 164923. https://doi.org/10.1016/j.ijleo.2020.164923

2.3 几种定位方法:基于全局光流的无人机速度估计

传感器包括一个下视单目摄像机，IMU，和超声波模块。通过计算连续图像的全局光流来实现像素运动估计。针对不同的表面情况设计了不同的算法。提出了一种基于特征的纹理表面全局光流估计方法，并将基于图像金字塔的方法推广到低纹理环境中。然后，结合根据光流、惯性数据和高度估计的像素运动，采用基于视觉的速度估计方法，给出可视化观测结果。最后，利用经典的卡尔曼滤波融合所有可用信息并给出速度估计。具体方法如下。



2.3 几种定位方法:基于全局光流的无人机速度估计

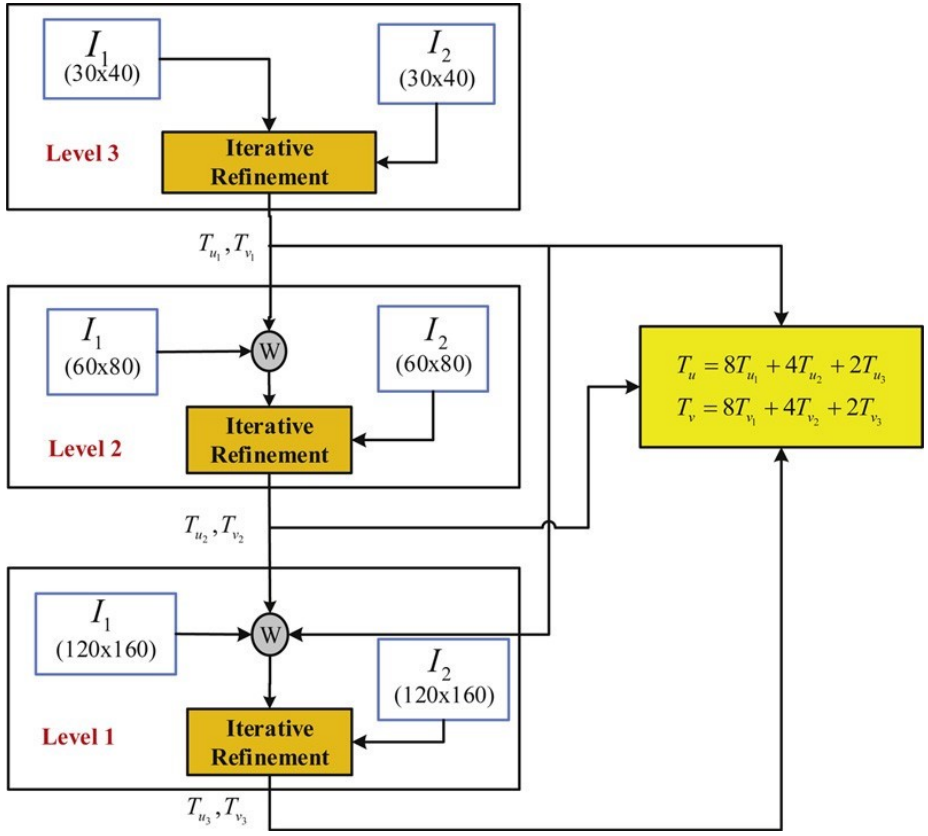
1.1 基于特征的纹理表面的全局光流

在我们的工作中，我们利用了面向快速旋转BRIEF (ORB)关键点描述符

$$T_u = Median(x_{I_2}(i) - x_{I_1}(i))$$
$$T_v = Median(y_{I_2}(i) - y_{I_1}(i))$$

1.2 基于图像金字塔的低纹理表面的全局光流

对于纹理较少、特征点较少的表面，上述基于特征的全局光流方法可能失效。我们使用Lucas和Kanade光流、迭代细化和图像金字塔来估计仿射运动模型参数，从而得到平面低纹理场景的全局光流。



2.基于视觉的像素运动速度估计

采用针孔相机模型，通过相机参数矩阵描述相机图像中三维场景与二维投影之间的关系。考虑三维空间中的静态点$\boldsymbol{p_e} = [\boldsymbol{x_e}, \boldsymbol{y_e}, \boldsymbol{z_e}]^{\boldsymbol{T}} \in \mathbb{R}^{\boldsymbol{3}}$，它在二维图像平面的投影是$\overline{\boldsymbol{p}}_{\boldsymbol{c}} = [\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{1}]^{\boldsymbol{T}} \in \mathbb{R}^{\boldsymbol{3}}$，他们的关系是

$$\mathbf{p}_e = z_c \mathbf{R}_c^e \overline{\mathbf{p}}_c - \mathbf{T}_e$$

如果连续两帧间摄像机的运动较小，则用世界帧表示摄像机的瞬时速度为

$$\mathbf{v}_{e(k)} = \frac{z_{c(k-1)} \mathbf{R}_{c(k-1)}^e \overline{\mathbf{p}}_{c(k-1)} - z_{c(k)} \mathbf{R}_{c(k)}^e \overline{\mathbf{p}}_{c(k)}}{\delta t}$$

3 信息融合

运动方程是惯性系下刚体运动连续时间模型的离散化得到，模型线性且直观简单，给出如下

$$\mathbf{v}_b = (\mathbf{f}_b - \mathbf{b}_a) + (\mathbf{R}_b^e)^T \mathbf{g}_l$$

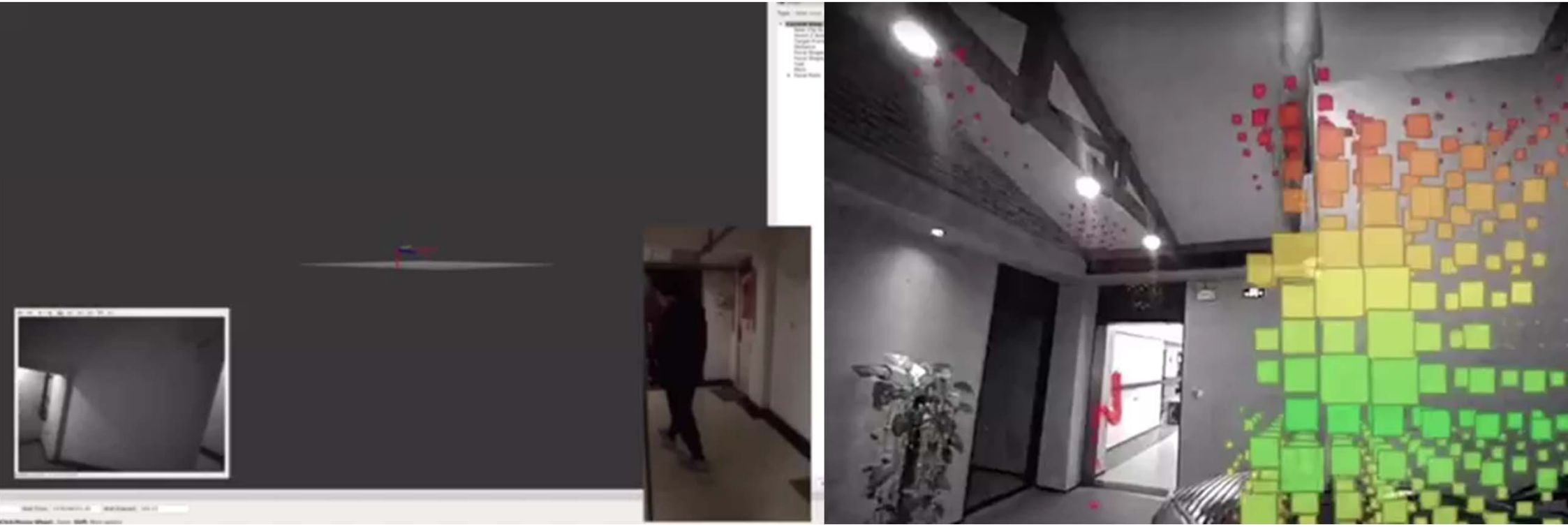
离散域的状态转移方程

$$\mathbf{x}_k = \mathbf{A}\mathbf{x}_{k-1} + \mathbf{u}_k + \mathbf{w}_k$$

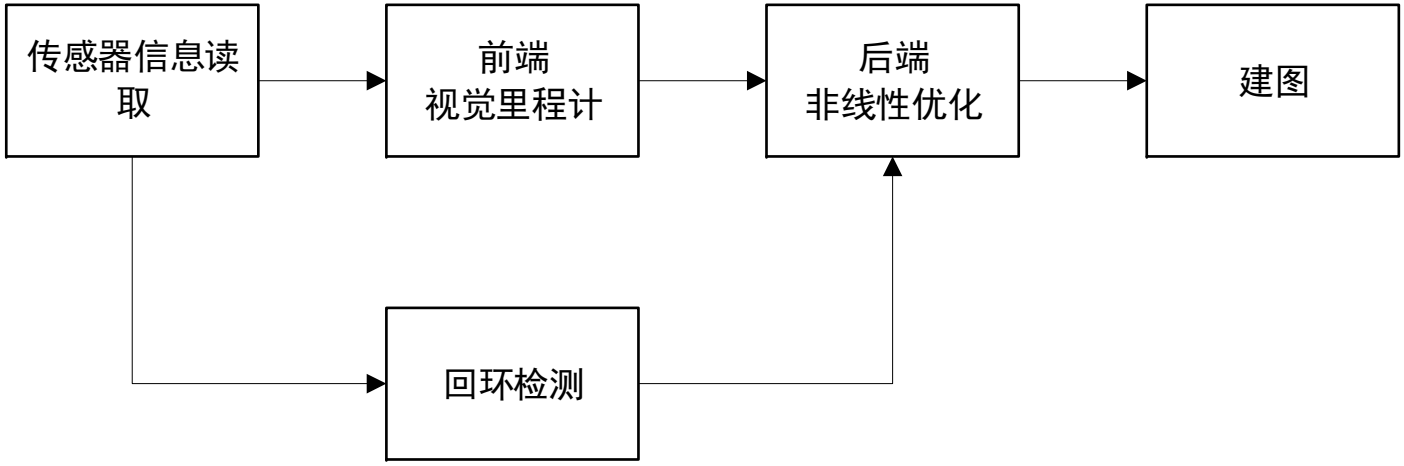
其中$\boldsymbol{A} \in \mathbb{R}^{7\times7}$是状态转移矩阵，$\boldsymbol{u_k} \in \mathbb{R}^{7}$是控制矩阵。具体形式为

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 0 & -\delta t & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & -\delta t & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & -\delta t \\ 0 & 0 & -\delta t & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \mathbf{u}_k = \begin{bmatrix} (f_{\mathrm{bx}} + r_{31}g)\delta t \\ (f_{\mathrm{by}} + r_{32}g)\delta t \\ (f_{\mathrm{bz}} + r_{33}g)\delta t \\ (f_{\mathrm{bz}} + r_{33}g)\dfrac{\delta t^2}{2} \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

2.3 几种定位方法: VIO/SLAM



2.3 几种定位方法: VIO/SLAM



前端视觉里程计（Visual Odometry, VO）负责图像处理部分。后端优化主要为通过不同时刻测量的相机位姿，及回环检测信息对其优化，得到全局一致的轨迹和地图。回环检测判断机器人是否到达过先前的位置，若有回环后端会进行处理。最后根据估计的轨迹建立与任务要求对应的地图。

2.3 几种定位方法: VIO/SLAM

主流视觉惯性里程计（Visual Inertial Odometry, VIO）框架比较

- 几乎没有松耦合的VIO方案
- 紧耦合主要分为滤波和优化，优化计算量大但精度高
- ROVIO在精度和效率平衡中的综合性能较好

VIO框架	耦合方案	后端方案	前端	视觉误差	初始化	回环	精度	效率
MSCKF	紧耦合	滤波EKF	fast+光流	重投影	静止	无	较差	较快
ROVIO	紧耦合	滤波IEKF	fast+光度	光度	静止	无	一般	较快
OKVIS	紧耦合	优化	Harris+BRISK	重投影	静止	无	较好	最慢
VINS	紧耦合	优化	Harris+光流	重投影	动态	有	最好	较慢

2.3 几种定位方法: VIO/SLAM

主流视觉惯性里程计（Visual Inertial Odometry, VIO）框架比较

- 几乎没有松耦合的VIO方案
- 紧耦合主要分为滤波和优化，优化计算量大但精度高
- ROVIO在精度和效率平衡中的综合性能较好

VIO框架	耦合方案	后端方案	前端	视觉误差	初始化	回环	精度	效率
MSCKF	紧耦合	滤波EKF	fast+光流	重投影	静止	无	较差	较快
ROVIO	紧耦合	滤波IEKF	fast+光度	光度	静止	无	一般	较快
OKVIS	紧耦合	优化	Harris+BRISK	重投影	静止	无	较好	最慢
VINS	紧耦合	优化	Harris+光流	重投影	动态	有	最好	较慢

2.4 总结

- 人类80%信息来源于视觉
- 这里主要考虑视觉伺服的视觉工作，因此这里没有讲：角点检测、光流、地图构建等

目标检测与跟踪	定位	地图构建
数字图像处理 • 灰度变化与滤波、图像分割、形态学处理、角点检测		



谢 谢！